Delivery Atlas

Documento técnico para el equipo

Destinatarios: Jere, Anuar y Echenique. **Revisión del código:** 14 de octubre de 2026. **Base revisada:** commit `5feb407`. Las mediciones y capturas conservadas corresponden al 7 de octubre; este documento no afirma que los servicios estén encendidos hoy.

Qué estamos construyendo

Una infraestructura analítica para una plataforma ficticia de comida a domicilio, inspirada en Uber Eats y Glovo. Recibimos acciones de navegación, estados de pedidos y posiciones de repartidores; ClickHouse conserva el histórico y alimenta un dashboard.

Hay dos partes que se complementan: el laboratorio contiene datasets públicos de gran volumen; Delivery Atlas tiene nuestro propio contrato de eventos y un escenario operativo simulado de Madrid. Sus identificadores y poblaciones permanecen separados.

Qué está implementado

- Un nodo ClickHouse, dos bases lógicas y almacenamiento persistente en Docker.

- API Python, validación, cola SQLite, publicación por lotes y control de reintentos.

- Generadores de navegación y GPS, catálogo ficticio y estados de pedidos.

- Dashboard con tracking, métricas, navegación e inventario de fuentes.

- Web compartida en Vercel, conectada al ordenador de Jere por un gateway de lectura y un túnel temporal.

La miniapp comercial completa, su checkout conectado a un pedido y la instalación comprobada en otro ordenador siguen pendientes. Los datos del tracking son ficticios; no tenemos acceso a la telemetría interna de Uber Eats o Glovo.

Cómo leer este documento

Si quieres entender...	Lee primero
La idea y las tecnologías	1-2: arquitectura y componentes
Cómo llega un registro a la base	3-7: ingesta, SQL y esquemas
Qué datos estamos mostrando	8-11: simulación, fuentes y panel
Cómo ejecutarlo o compartirlo	12-14: Vercel, arranque y consultas
Qué está demostrado y dónde trabajar	15-16: resultados y mapa del código

Repositorio: https://github.com/Gere2/analitica-reparto-clickhouse. Demo publicada el 7/10: https://delivery-atlas-demo.vercel.app. Su actualización depende del laboratorio local.

1. Arquitectura: quién hace cada trabajo

Camino de un evento nuevo

```
Generador o acción manual
        |
Validación Python -> SQLite: evento pendiente
        |
Publicador por lotes -> ClickHouse: histórico + agregados
        |
API de lectura -> dashboard: métricas y posiciones
```

El simulador integrado llama directamente a la misma validación y cola. Un cliente externo entra por HTTP. Las cargas públicas del laboratorio utilizan sus scripts ETL y escriben en otra base lógica.

Servicios y direcciones

Componente	Función y dirección
`clickhouse`	Un servidor ClickHouse. Dentro de Docker: `clickhouse:8123`; desde el ordenador: `127.0.0.1:8124`.
`atlas`	API, simulador y publicador en un mismo proceso Python. Web/API: `127.0.0.1:8001`.
`share-gateway`	Servicio adicional de lectura para compartir. Puerto 8080 interno; no se publica un puerto del host.
`share-tunnel`	Conexión saliente con Cloudflare hacia el gateway. Solo se arranca para compartir.
Laboratorio histórico	`dashboard.py`, ejecutado aparte en `127.0.0.1:8000`. No forma parte del proceso Atlas.

Un nodo es una instancia del servidor de base de datos. Los cuatro servicios de la compartición no equivalen a cuatro nodos ClickHouse: seguimos teniendo uno.

Almacenamiento persistente

- Volumen `clickhouse_data`: tablas, partes y estado de ClickHouse.

- Volumen `atlas_queue`: SQLite en `/state/outbox.sqlite`.

- Directorio local `data/`: archivos fuente, manifiestos y configuración privada de compartición; ignorado por Git.

Los volúmenes permiten conservar datos entre reinicios. No aportan por sí solos réplica, copia de seguridad o protección ante pérdida del disco. `delivery` y `delivery_atlas` comparten el mismo servidor y sus recursos.

Archivos de referencia: `compose.yaml`, `compose.share.yaml`, `Dockerfile.atlas`, `Dockerfile.share` y `dashboard.py`.

2. Tecnologías utilizadas y por qué

Tecnología	Uso concreto en este proyecto
ClickHouse 25.8	Almacenar eventos masivos y consultar agrupaciones, históricos y últimos estados. Imagen `clickhouse/clickhouse-server:25.8`; versión observada en la medición: 25.8.33.6.
SQL	Definir tablas, particiones, vistas y consultas analíticas. Utilizamos SQL aunque la asignatura agrupe esta tecnología con bases NoSQL.
Python 3.12 en Docker	API, validación, publicador, simulación y gateway. La ruta Atlas utiliza la biblioteca estándar, sin instalar un framework web.
SQLite	Cola persistente local y registro de IDs aceptados. Su papel es transportar eventos hacia ClickHouse.
Docker y Compose	Reproducir los servicios, red interna, configuración y volúmenes. Atlas se activa con el perfil `atlas`.
HTML, CSS, JavaScript y SVG	Interfaz, consultas HTTP, tablas, barras y plano de coordenadas. No requiere React ni una API externa de mapas.
Node.js y Vercel	Preparar la web compartida y ejecutar `api/proxy.js` por petición. La función reenvía lecturas al laboratorio.
Cloudflare Tunnel	Conectar temporalmente Vercel con el gateway local mediante HTTPS, sin publicar el SQL de ClickHouse. Imagen fijada: `2026.3.0`.
Git y GitHub	Compartir código, documentación, SQL, capturas y resultados pequeños. Los datasets masivos no están en el repositorio.
ReportLab y Poppler	Generar y revisar este PDF. Son herramientas de documentación, no dependencias de la demo.

Por qué ClickHouse encaja

Nuestro trabajo combina muchas inserciones de eventos y lecturas que cuentan, agrupan y comparan períodos. Organizamos los datos con motores de la familia MergeTree y preparamos estados agregados para el dashboard. El objetivo es analítica, no resolver cobros o asignaciones transaccionales.

ClickHouse no mide clics por sí solo: **nosotros registramos eventos y la base permite analizarlos**. Una posición GPS o una fila de catálogo tampoco se convierte en un clic por almacenarse allí.

Existe una implementación inicial en `app/` con Flask 3.1.3 y `clickhouse-connect` 1.9.0. No es la API activa de Atlas. Esta última conecta por HTTP con `urllib.request`; devuelve resultados JSON e inserta en formato `JSONEachRow`.

También conservamos `docker-compose.yml`: configuración inicial de esa app y Grafana 12.3.0, con el plugin `grafana-clickhouse-datasource` y conexión nativa al puerto 9000 de la base `reparto`. Servía para visualizar consultas mediante Grafana y separar usuarios de escritura/lectura. La ruta actual del proyecto usa `compose.yaml` y nuestro dashboard web; arrancar aquel fichero no reproduce Atlas.

Configuración local: `clickhouse-config.xml` limita el pool de merges a cuatro; el cliente Atlas fija `max_threads=4`, inserciones síncronas y espera de final de consulta. Son ajustes del equipo de demostración, no una medición de capacidad máxima del motor.

3. Ingesta, cola y reintentos

Recorrido exacto

1. El cliente crea un UUID por evento, su hora y su contenido.
2. `atlas/server.py` recibe el JSON y llama a `contract.py` o `operations.py`.
3. `Store.accept()` abre una transacción SQLite con `BEGIN IMMEDIATE`, comprueba IDs y guarda el lote.
4. La API responde cuando la cola ha aceptado el evento. **202 no significa que el dashboard ya lo pueda consultar.**
5. `Publisher` lee hasta 5000 pendientes, agrupa por tabla e inserta con el cliente de ClickHouse.
6. Solo después marca esas filas como publicadas. Las vistas materializadas procesan las inserciones y la API consulta los resultados.

Qué conserva SQLite

Estructura	Contenido
`outbox`	Secuencia local, `event_id` único, SHA-256 del JSON canónico, payload con `ingested_at` y bandera `published`.
`counters`	Aceptados, duplicados, peticiones, conflictos y errores observados.
`metadata`	Última configuración/run del simulador para recordarlo al reiniciar.

Se usa WAL, `synchronous=FULL` y una conexión persistente protegida por `RLock`. La caché de páginas se configura a 32 MiB. Estas medidas evitan reabrir la conexión en cada tick; no sustituyen un backup.

Contrato de entrega

Situación	Resultado
ID nuevo y contenido válido	Se guarda; respuesta 202.
Mismo ID y mismo contenido	Se reconoce el reintento; 200 si el lote solo contiene duplicados.
Mismo ID con otro contenido	Conflicto 409; se rechaza el lote SQLite.
Cola con 200000 pendientes	503 con indicación de reintento. No se confirma como aceptado.
Contrato incorrecto / cuerpo excesivo	400 / 413. Máximo 5000 eventos y 5 MiB por petición.

Si se pierde una respuesta, reenviar **el mismo ID, hora y contenido**. Crear un nuevo UUID convertiría el reintento en otro evento.

Un lote mezclado puede necesitar tres INSERTs en ClickHouse. No forman una transacción conjunta. Si el segundo falla, el lote queda pendiente y se reenvía: puede generar copias físicas, que las lecturas lógicas resuelven. No prometemos entrega física exactamente una vez.

4. Cómo hemos diseñado ClickHouse

Separación por función

Base	Qué contiene
`delivery`	Fuentes públicas, sus agregados, replay histórico y tabla de ensayo CRUD.
`delivery_atlas`	Navegación propia, catálogo ficticio, posiciones, estados y vistas de lectura.

Motores y objetos

Elemento	Qué significa en nuestra implementación
`MergeTree`	Histórico tipado del laboratorio. Organiza partes de datos para las consultas.
`ReplacingMergeTree(version)`	Conserva la versión más reciente de filas con la misma clave de ordenación cuando se fusionan. En Atlas se usa `ingested_at` o `version`.
`AggregatingMergeTree`	Guarda estados intermedios de funciones de agregado; se combinan al consultar.
`SummingMergeTree`	Agregados simples del laboratorio anterior. No es el contador exacto elegido para los reintentos de Atlas.
Vista materializada incremental	Procesa bloques insertados y escribe estados en otra tabla. No es un hilo que consulte continuamente toda la base.
Vista de lectura	Consulta guardada que finaliza/combina estados cuando la API la lee.

Referencias: ReplacingMergeTree, AggregatingMergeTree y vistas materializadas.

Partición, orden y unicidad

Las tablas de eventos Atlas se particionan por mes. Su `ORDER BY` organiza los datos para las lecturas y define las claves sobre las que actúa el motor de reemplazo; **no impone una restricción UNIQUE**. La identidad y el contenido inmutables se controlan primero en SQLite.

Las fusiones son posteriores a la inserción: `count()` sobre filas físicas puede diferir de una lectura lógica. Usamos `FINAL` para resolver copias al leer detalle, `uniqExact` para contar IDs y `argMaxMerge` para obtener el último estado.

`CREATE ... IF NOT EXISTS` permite arrancar sin recrear objetos existentes. No es un sistema de migraciones: editar el SQL no transforma automáticamente una tabla ya creada ni rellena agregados con su histórico. Una evolución del esquema necesita un procedimiento explícito.

5. Navegación: contrato y agregado diario

Una fila de `delivery_atlas.app_events` **representa una acción lógica.** No representa una posición, un usuario único o una venta cobrada.

Columna o grupo	Tipo SQL y significado
`schema_version`	`UInt16`; versión admitida: 1.
`event_id`, `session_id`	`UUID`; identidad del evento y agrupación de la sesión.
`event_time`, `ingested_at`	`DateTime64(3, 'UTC')`; hora del evento y aceptación en la cola.
`event_date`	`Date MATERIALIZED toDate(event_time)`; calculada por ClickHouse.
`event_type`, `section`, `city_id`	`LowCardinality(String)`; acción, pantalla/sección y ciudad.
`user_id`, `restaurant_id`, `product_id`	`Nullable(String)`; referencias conocidas; `null` cuando no se conocen.
`source`	`LowCardinality(String)`; generador o cliente que produjo el registro.
`source_kind`	`Enum8`: `synthetic`, `demo_live`, `public_historical` o `historical_replay`.
`source_record_id`	`Nullable(String)`; referencia a la observación histórica original.
`simulation_run_id`	`Nullable(UUID)`; identifica una ejecución sintética.

La API exige los campos comunes, valida UUIDs y fechas con zona horaria, convierte a UTC y rechaza campos desconocidos. Un evento sintético necesita run; uno histórico necesita referencia original. Producto y carrito necesitan además `restaurant_id` y `product_id`.

Acciones admitidas: `search`, `filter_applied`, `restaurant_opened`, `product_opened`, `product_impression`, `cart_added`, `order_submitted`. El contrato permite más acciones de las que muestra el generador básico.

Los cuatro objetos de navegación

```
app_events -> app_daily_rollup_mv -> app_daily_rollup
                                          |
                                  app_daily_counts
```

`app_daily_rollup` agrupa por procedencia, fecha, ciudad, sección y acción. Guarda `AggregateFunction(uniqExact, UUID)`; la materializada produce `uniqExactState(event_id)` y la vista combina con `uniqExactMerge(event_ids)`.

Así un reenvío con el mismo ID y dimensiones no suma dos veces dentro del grupo. No sumamos simplemente `count()` de cada INSERT. El conjunto exacto tiene coste de memoria y combinación, que aparece en las mediciones.

Clave de ordenación del detalle: `(source_kind, event_date, city_id, event_type, event_time, session_id, event_id)`. **Fuente de verdad:** `sql/atlas.sql` y `atlas/contract.py`.

6. Posiciones y pedidos: columnas utilizadas

Hay dos tablas de hechos separadas. `courier_positions` registra una posición; `order_events` registra una foto completa de un pedido cuando cambia de estado.

Columnas comunes a las dos tablas

Campos	Tipo y uso
`schema_version`	`UInt16`; contrato v1 validado por Python.
`event_id`, `simulation_run_id`	`UUID`; evento y ejecución.
`courier_id`, `order_id`	`UUID`; repartidor y pedido de la simulación.
`event_time`, `ingested_at`	`DateTime64(3, 'UTC')`; tiempo del evento y aceptación.
`city_id`, `source`	`LowCardinality(String)`; ciudad y productor.
`source_kind`	`Enum8('synthetic'=1)`; estas rutas solo admiten telemetría ficticia.

Campos exclusivos del GPS

Campos	Tipo y uso
`latitude`, `longitude`	`Float64`; coordenadas. Rangos validados: ±90 y ±180.
`speed_kmh`, `accuracy_m`	`Float32`; velocidad y precisión simuladas. La API admite 0-150 km/h y 0-10000 m.

Campos exclusivos del estado del pedido

Campos	Tipo y uso
`restaurant_id`	`String`; negocio ficticio de recogida.
`status`	`LowCardinality(String)`; estado admitido por el contrato.
`amount_minor`, `currency`	`UInt32` y `FixedString(3)`; céntimos enteros y `EUR`.
`pickup_lat`, `pickup_lon`, `dropoff_lat`, `dropoff_lon`	`Float64`; coordenadas ficticias de recogida y entrega.
`order_created_at`, `promised_delivery_at`	`DateTime64(3, 'UTC')`; creación y plazo ficticio.

Estados: `created`, `preparing`, `picked_up`, `en_route`, `delivered`, `cancelled`. La API comprueba que creación no sea posterior al evento y que el plazo no sea anterior a creación; no implementa una máquina transaccional que prohíba todas las transiciones de negocio.

Ambas tablas usan `ReplacingMergeTree(ingested_at)` y partición mensual. GPS ordena por `(simulation_run_id, city_id, courier_id, event_time, event_id)`; pedidos, por `(simulation_run_id, city_id, order_id, event_time, event_id)`.

`ingested_at` lo añade la cola. `_stream` es una marca interna de enrutamiento que se retira antes del INSERT; el cliente no debe enviarla. Código y DDL: `atlas/operations.py` y `sql/atlas_operations.sql`.

7. Obtener el último estado sin perder el histórico

GPS

```
courier_positions -> courier_latest_mv
                            |
                    courier_latest_state
                            |
                     courier_latest
```

La materializada guarda `argMaxState` en una tabla `AggregatingMergeTree`. La vista `courier_latest` combina esos estados con `argMaxMerge`. La clave de agrupación es ejecución, ciudad y repartidor.

El criterio de comparación es `tuple(event_time, event_id)`. Gana la hora de evento más reciente; ante empate, el UUID resuelve de forma determinista. Latitud, longitud, velocidad, pedido y horas se guardan juntos en una tupla, para que el resultado proceda de la misma fila.

Ejemplo: llega tarde una posición antigua

Evento	Hora del evento	Hora de llegada	Resultado
A	12:00:02	12:00:03	Posición seleccionada inicialmente.
B	12:00:00	12:00:04	Se conserva en histórico; no sustituye a A.

Ordenar solo por llegada movería al repartidor hacia atrás. Nuestro criterio toma el tiempo del evento. Referencia: argMax.

Pedidos

`order_events -> order_latest_mv -> order_latest_state -> order_latest` aplica el mismo patrón, agrupando por ejecución, ciudad y pedido. Conserva la foto completa: estado, repartidor, negocio, importe, coordenadas y horas.

Los objetos `*_state` almacenan estados agregados, no un documento de usuario ni una fila de GPS directa. Las vistas `*_latest` son las que consume la API. El histórico sigue disponible en las tablas de hechos.

Relaciones y consistencia

El simulador crea referencias coherentes entre repartidor, pedido y negocio ficticio. No hay claves foráneas que ClickHouse esté comprobando en este modelo. Los IDs no identifican personas de otros datasets.

La API consulta posiciones, pedidos, métricas, ritmo y hora mediante SELECTs separados. No son una fotografía transaccional única: pueden existir diferencias breves durante la publicación. Tampoco existe réplica o sharding configurado. Separar run impide mezclar ejecuciones en la lectura de estados.

8. Simulación y catálogo: qué hemos inventado

Flota en tiempo de reloj

`atlas/simulation.py` contiene `Scenario` (datos del escenario) y `Simulator` (hilo que genera ticks). Por defecto: Madrid, 12 repartidores, semilla 42 e intervalo solicitado de dos segundos. Admite 1-50 repartidores e intervalos de 1-10 segundos.

Cada tick produce GPS; solo los cambios de estado producen fotos de pedidos. Los ciclos duran 160 s y comienzan escalonados; algunos pedidos empiezan ya avanzados, por lo que su primer histórico puede ser parcial. Aproximadamente un 12% de los ciclos se configura para cancelar. Estos parámetros describen el generador, no la demanda real de Madrid.

Las coordenadas se interpolan entre origen, recogida, un punto intermedio y entrega. La siguiente ruta parte del destino anterior, o de recogida tras cancelación. Velocidad aproximada y precisión de 5 m también son simuladas; no se han observado ni validado contra calles reales.

La generación usa semilla y UUID5 para reproducir el escenario cuando se conservan run y tiempos. Iniciar otra flota crea un nuevo run. Parar conserva el histórico; reiniciar la API recuerda el último run pero **no reactiva automáticamente el simulador**.

Dos generadores de navegación distintos

- `scripts/atlas.py`: carga grande reproducible, tres ciudades y secuencias de cinco acciones completas. No sirve para estimar abandono real.
- Simulador integrado: sesiones sintéticas con abandono posible entre pasos. Se generan aparte de los pedidos; no hay enlace comercial entre ese checkout y la entrega.
- `scripts/operations.py`: carga histórica de GPS/estados por HTTP con tiempos fijados. No genera una flota que se mueva ahora ni envía sus clics auxiliares.

Catálogo: 12 negocios y 36 productos

`atlas/catalog.py` lo siembra al arrancar si no existen las cantidades esperadas. `demo_restaurants` guarda ID, nombre, ciudad, coordenadas, origen ficticio y `version`. `demo_products` guarda ID, negocio, nombre, `price_minor`, EUR, `price_kind`, `name_kind`, `source_table`, `source_row` y `version`.

Si está cargado Glovo Madrid, toma 36 nombres y sus filas de origen; en otro ordenador usa nombres de respaldo. **Precio, ubicación y asignación al negocio siguen siendo ficticios.** `name_kind` conserva la procedencia del nombre. El importe del pedido se genera de forma independiente: no es la suma de una cesta real del catálogo.

DDL del catálogo: claves `restaurant_id` y `product_id`, motor `ReplacingMergeTree(version)`. La futura miniapp podrá reutilizar `GET /api/demo-catalog`.

9. Fuentes públicas: volumen y significado

Recuentos documentados el **7/10/2026**, previos a nuevas inserciones. No se han vuelto a contar para redactar este PDF. La auditoría de columnas está en `docs/auditoria-esquemas.json`.

Fuente / tabla en `delivery`	Filas verificadas	Qué representa una fila
OTTO / `otto_archive`	216.716.096	Evento de navegación: clic, carrito o pedido.
REES46 / `rees46_events`	411.709.736	Vista, carrito o compra de comercio electrónico.
Ele.me / `eleme_samples`	2.170.299	Muestra de recomendación con etiqueta `clicked`.
Glovo / `glovo_products`	2.887.444	Producto del catálogo publicado; no un clic.
TRD / `orders`	1.068.495	Pedido histórico del mismo corpus TRD/Meituan.
TRD / `clicks`	4.021.488	Paso de una secuencia de restaurantes registrados.
TRD / `click_summary`	517.353	Resumen de una secuencia; no un evento individual.
TRD / `restaurants`, `foods`	29.072 / 179.778	Dimensiones de negocio y producto.
TRD / `order_items`	3.445.180	Relación entre pedido y producto de ese corpus.
`otto_replay`	1.200.000	Inserción de un evento antiguo reproducido.
`otto_scale_benchmark`	1.083.580.480	Fila derivada de OTTO para estrés; no nueva observación.

Campos útiles que conservamos

OTTO: sesión, artículo, hora, acción y posición. REES46: hora, acción, producto, categoría, marca, precio, usuario y sesión. Ele.me: archivo/fila, usuario, tienda, artículo, ciudad anónima, contexto horario, etiqueta y características originales en JSON. Glovo: fila, país, ciudad, negocio, producto, sección y descripción.

TRD permite relacionar pedidos, clics y productos usando sus propios IDs y fechas. `price_band` es un intervalo de precio, no un importe exacto. Sus secuencias de clics no proporcionan el GPS de los repartidores.

Glovo tiene 415.137 productos de España y 68.575 de Madrid en la carga documentada. La muestra del panel no demuestra que esos productos o negocios sigan activos hoy.

No unimos los IDs de OTTO, REES46, Ele.me, Glovo y Atlas. Una suma de todas sus filas no significa usuarios, clics o pedidos de una única plataforma. Criteo, MT-LIFT y censo de Madrid aparecen en propuestas antiguas, pero no en las tablas activas auditadas.

10. Cómo se cargaron los históricos

Estas rutas pertenecen al laboratorio. La demo pequeña de Atlas puede arrancar sin descargar sus archivos masivos.

Ruta de código	Transformación y uso
`scripts/download.py`, `prepare.py`, `load.py`	Descargar/preparar TRD, convertir TSV a campos tipados y cargar las tablas del corpus. `sql/schema.sql` define el esquema.
`scripts/otto.py`	Gestionar el archivo OTTO, cargar eventos, verificar recuentos y reproducirlos en `otto_replay`.
`scripts/rees46.py`	Leer CSV comprimidos por mes, validar cabecera/tamaño y publicar lotes con control de carga parcial.
`scripts/eleme.py`, `sql/eleme.sql`	Validar el formato de 39 campos, cargar staging y publicar archivos completos con `MOVE PARTITION`.
`scripts/glovo.py`	Transformar el catálogo FooDI-ML a `glovo_products`, conservando fila y atributos de origen.
`scripts/rollup.py`, `benchmark.py`	Preparar agregados OTTO y comparar consultas del laboratorio.
`scripts/scale.py`	Crear el escenario derivado de estrés. No aporta clics observados adicionales.
`dashboard.py` y páginas de `web/`	Leer el laboratorio y mostrar OTTO, REES46, Ele.me, Glovo, TRD y volumen.

Controles de rigor

Se conservan informes, manifiestos, huellas y recuentos en las rutas de cada importador. OTTO se contrastó con la huella y los recuentos del archivo; REES46 controla cada mes; Ele.me conserva `source_id`, archivo y fila, además de las características originales.

En Ele.me no basta con tener 39 columnas: su correspondencia debe verificarse antes de confirmar el layout. La carga revisada es una parte del dataset anunciado, no todos los datos de la plataforma.

Mover partes con `MOVE PARTITION` no activa una vista materializada de inserción como si se insertaran filas nuevas. Tampoco se crea automáticamente un agregado retrospectivo al añadir una materializada a un histórico existente.

El replay conserva la hora histórica y añade la hora de ingesta. **Que entre en la base durante la demo no lo convierte en un clic de un usuario actual.**

Los archivos fuente grandes permanecen fuera de Git y de la subida de Vercel. El equipo comparte el código y las instrucciones; volver a obtener una fuente requiere seguir su acceso y condiciones. Las guías de cada dataset están en `docs/DATOS.md`, `GLOVO.md` y `ELEME.md`.

11. Dashboard y API: de dónde sale cada dato

El panel usa `web/atlas-dashboard.html`, `.css` y `.js`. La API sirve web y JSON en el mismo origen; el navegador no necesita credenciales directas de ClickHouse.

Endpoint local	Qué devuelve o hace
`GET /api/operations`	Posiciones, lista de pedidos, métricas y GPS por intervalos de 5 s. Usa el último run si no se indica otro.
`GET /api/track`	Hasta 240 posiciones lógicas del repartidor/run, ordenadas para dibujar el recorrido.
`GET /api/simulation`	Estado, configuración y último run del simulador.
`GET /api/status`	Pendientes/publicados y fases del publicador local.
`GET /api/summary`	Navegación agregada; filtros de fecha, ciudad, procedencia y acción.
`GET /api/recent`	Últimos 30 eventos de navegación globales.
`GET /api/inventory`	Metadatos de tablas, filas físicas estimadas y disco.
`GET /api/catalog`	Muestra Glovo Madrid si está cargada.
`GET /api/demo-catalog`	Los negocios/productos de la demo, con procedencia por campo.
`POST /api/events`, `/api/operations`	Aceptación en la cola de navegación o telemetría.
`POST /api/simulation/start`, `/stop`	Control local de la flota. Inicio acepta `fleet`, `interval`, `seed`; parada recibe `{}`.
`GET /health`	Consulta `SELECT 1` y estado del publicador para el healthcheck.

Lecturas del panel

- Activos, entregados, cancelados e importe: último estado de cada pedido. Fuera de plazo: pedidos todavía activos cuyo plazo ficticio ha pasado.
- Lista limitada a 200 pedidos; las métricas consultan el conjunto del run. `truncated` avisa del límite conservador.
- El plano SVG proyecta latitud/longitud. La línea utiliza solo las posiciones recibidas del pedido actual; no calcula rutas de calles.
- GPS por 5 s: `uniqExact(event_id)` en los dos últimos minutos. Navegación: recuentos por acción, no conversión de esos pedidos.

Tracking se vuelve a consultar tras completar el ciclo anterior y esperar 2 s; navegación, aproximadamente cada 10 s. No se solapan ciclos de polling. El inventario se carga al abrir la página.

El panel local permite registrar una búsqueda manual. `event_time` es la hora de acción; `ingested_at` mide aceptación en API. Su diferencia no mide publicación completa. La antigüedad de la posición muestra el retraso visible; si falla una consulta, se conservan los datos anteriores y se avisa.

12. Compartición: Vercel y Cloudflare

Camino de una consulta remota

```
Compañero -> Vercel: HTML + JavaScript + función GET
         -> Cloudflare: URL HTTPS temporal
         -> share-gateway: token y rutas permitidas
         -> API Atlas local -> ClickHouse local
```

Vercel aloja la interfaz y el proxy por petición. El servidor ClickHouse, el archivo SQLite y los hilos permanentes siguen en Docker en el ordenador de Jere. Si el ordenador duerme o se detiene el túnel, la web permanece publicada y deja de recibir actualizaciones.

Archivos y protección del acceso

`scripts/build_shared.mjs` copia los tres assets, marca el modo de lectura y retira controles de inicio/parada e inserción. `api/proxy.js` solo acepta GET y rutas permitidas; utiliza `ATLAS_SHARE_ORIGIN` y `ATLAS_SHARE_TOKEN` desde variables del servidor. El token no se añade al JavaScript del navegador.

`atlas/share.py` exige el token, valida UUIDs/parámetros, limita la respuesta a 2 MiB y rechaza POST, PUT, PATCH y DELETE. Reduce los campos de estado/simulación y no devuelve errores internos detallados.

Rutas públicas admitidas: `simulation`, `operations`, `status`, `track`, `summary`, `inventory`, `catalog`. La compartición limita summary a Madrid y a `synthetic`/`demo_live`; no expone todos los filtros de la API local. Tampoco publica `recent` o `demo-catalog`.

Caché del gateway	Duración
Seguimiento, estado y recorrido	1,5 s
Navegación	10 s
Inventario y catálogo	60 s

Máximo 128 claves, con bloqueos separados por endpoint para que inventario no bloquee GPS. La función corta la espera a los 12 s; su duración máxima configurada en Vercel es 30 s. Estas cachés pueden añadir demora.

Condiciones de la demo compartida

El enlace es visible para quien lo tenga. Transporta datos ficticios, agregados y catálogo público por Vercel/Cloudflare. ClickHouse mantiene su puerto local; la API de escritura tampoco se abre por el túnel.

La URL `trycloudflare.com` cambia al crear otro túnel: hay que actualizar `ATLAS_SHARE_ORIGIN` y desplegar. Secretos en `data/sharing/` y variables sensibles de Vercel; código permitido en `.vercelignore`. Procedimiento completo: `docs/COMPARTIR-VERCEL.md`. No hay disponibilidad independiente del portátil.

El Compose actual usa credenciales de demostración locales; no implementa el reparto de roles de una aplicación de producción. Referencias: funciones de Vercel y Quick Tunnels de Cloudflare.

13. Arranque reproducible para compañeros

Requisitos y ruta pequeña

Docker con Compose v2 y Python 3.10 o posterior en el ordenador; los contenedores usan Python 3.12. Obtener el código del repositorio, incluyendo los cambios de la PR 1/rama `codex/actualizacion-clickstream-eleme` mientras no esté integrada.

Desde la carpeta del proyecto:

```
docker compose -f compose.yaml --profile atlas up -d --build --wait
docker compose -f compose.yaml --profile atlas ps
python3 -m unittest discover -s tests -v
python3 scripts/check_atlas.py
python3 scripts/check_operations.py
```

Abrir `http://127.0.0.1:8001/`. Iniciar la simulación; seleccionar un repartidor y comprobar hora, coordenadas y recorrido. Registrar una búsqueda manual y elegir la procedencia de acciones manuales para localizarla.

Las pruebas HTTP necesitan la API y ClickHouse; añaden filas de comprobación identificadas. Las unitarias prueban contratos/cola/rutas sin descargar históricos. En una máquina nueva el catálogo será ficticio y las fuentes públicas no aparecerán hasta cargarlas.

Cargas opcionales pequeñas

```
python3 scripts/atlas.py generate --events 10000 --batch 1000 --seed 42
python3 scripts/atlas.py status
python3 scripts/operations.py --positions 10000 --fleet 12 --seed 42
```

Esperar a `pending=0` antes de comprobar cargas o medir. Para reproducir navegación exactamente, guardar y reutilizar `--run-id`, `--start`, `--seed` y `--events`: sin fecha fija el generador elige otra. La carga GPS calcula un run determinista con fecha, semilla, flota y número de posiciones.

Logs, cambios y parada

```
docker compose -f compose.yaml --profile atlas logs --tail 60 atlas
docker compose -f compose.yaml --profile atlas stop
```

Repetir `up ... --build --wait` tras cambios de Python, SQL o web. El simulador deberá iniciarse de nuevo. Conservar los volúmenes; `down -v` elimina el almacenamiento de la base y la cola.

Compartir requiere preparar los archivos privados y arrancar el Compose adicional, según la guía de Vercel. **No lanzar el fichero de compartición en una instalación nueva antes de crear `data/sharing/gateway.env`.**

14. Consultas y diagnóstico que debéis conocer

Tres consultas para demostrar el modelo

```
-- Recuento lógico por tipo: lectura de conjuntos de IDs.
SELECT event_type, sum(events) AS events
FROM delivery_atlas.app_daily_counts
WHERE city_id = 'madrid' AND source_kind = 'synthetic'
GROUP BY event_type;

-- Posiciones de la última ejecución, usando su UUID real.
SELECT courier_id, latitude, longitude, event_time
FROM delivery_atlas.courier_latest
WHERE simulation_run_id = {run:UUID};

-- Motor y tamaño: metadatos, no recuento de usuarios.
SELECT database, name, engine, total_rows
FROM system.tables
WHERE database IN ('delivery', 'delivery_atlas');
```

El parámetro `{run:UUID}` necesita el run de `GET /api/simulation`; el cliente Python pasa `param_run`. Los filtros de lectura de Atlas se parametrizan en lugar de concatenar valores del usuario.

Leer un fallo por capas

Síntoma	Comprobación y significado
No aparecen repartidores	Ver `GET /api/simulation`; iniciar desde la demo local. Un reinicio recuerda datos pero no reactiva la generación.
Aceptado, pero aún no visible	Ver pendientes y fase en `/api/status`; 202 confirma SQLite, no publicación.
Pendientes crecen / error del publicador	Revisar salud de ClickHouse y logs Atlas. El publicador reintenta; la cola tiene un límite.
Fuente pública vacía en otro equipo	Los archivos masivos y volúmenes no viajan con Git. La simulación funciona igualmente.
Vercel deja de actualizar	Comprobar ordenador, Docker, túnel y URL de origen. Los controles pertenecen al panel local.
Conteos físicos distintos	Reintentos/fusiones pueden alterar filas físicas. Consultar `FINAL`, IDs únicos o vistas lógicas.

CRUD requerido por la asignatura

`python3 scripts/crud_demo.py` demuestra INSERT, SELECT, ALTER UPDATE y ALTER DELETE en `delivery.demo_events`, esperando la finalización de las mutaciones. La tabla es de ensayo, separada del flujo Atlas.

El script pausa temporalmente merges de tres tablas históricas grandes si existen y los reactiva en `finally`. Tenerlo en cuenta al usarlo durante cargas. Los estados de pedido de la aplicación se representan insertando otra foto; no se actualiza cada GPS con una mutación.

15. Evidencia, límites y relación con BD2

Estas son comprobaciones guardadas del 7/10/2026. No sustituyen una prueba nueva bajo carga ni una instalación en otro ordenador.

Comprobación	Resultado y alcance
Navegación Atlas	1.000.000 eventos sintéticos verificados por run. Envío a API/cola: 323,5853 s; no es throughput aislado de ClickHouse.
GPS histórico	100000 posiciones y 5965 estados verificados en ClickHouse. Envío de 105965 registros: 435,887 s con actividad concurrente.
Reintento y contenido	Reconocimiento de duplicados; conflicto 409 si cambia el contenido del mismo ID.
Recuperación	Evento pendiente recuperado tras paradas ordenadas y reinicio con volúmenes conservados. No prueba pérdida de disco.
Fallo entre INSERTs	Reenvío de lote mixto: 4 posiciones físicas / 2 lógicas; 2 pedidos lógicos. No hay transacción entre tablas.
Evento GPS antiguo	No sustituye a la posición de hora más reciente. Coordenada inválida: 400.
Lectura local de flota	Tres muestras sin carga histórica concurrente: antigüedad máxima 1,761-2,279 s; consultas 0,177-0,855 s.
Vercel	Siete rutas leídas; tiempos/posiciones avanzaron y una escritura devolvió 405. Se observó un 503 inicial recuperado. Última comprobación: 12 repartidores, antigüedad máxima 3,796 s en la consulta.

Un resultado que debemos explicar con honestidad

El agregado exacto Atlas devolvió el mismo resultado que el detalle, pero su mediana fue **0,703 s**, frente a **0,498 s** del detalle en tres consultas por alternativa. Leer menos filas no garantizó menos tiempo: combinar conjuntos exactos tiene coste. El resultado más rápido del agregado OTTO pertenece a otro esquema/consulta y no demuestra una mejora de Atlas.

Qué podemos presentar al profesor

Contexto y modelo columnar; motores/particiones/vistas; instalación Docker; Python conectado por HTTP; datasets públicos y sintéticos etiquetados; ingesta, dashboard y CRUD; mediciones con condiciones; explicación de replicación y sharding como ampliaciones, sin afirmar que los ejecutamos.

Qué queda por resolver

Miniapp comercial y enlace checkout-pedido; reproducción en otra máquina; retención/backup; pruebas sostenidas de demora con carga; evolución de esquema y, después, experimento con más nodos. La cola conserva payloads publicados y crece; no hay TTL de telemetría ni alta disponibilidad configurados.

Evidencia: `docs/evidencia-atlas.json`, `evidencia-carga-gps.json`, `evidencia-tracking.json`, `evidencia-frescura.json`, `evidencia-sharing.json`, `evidencia-vercel.json`. La carpeta `tests/` contiene 11 métodos de prueba al revisar el código; este documento no vuelve a ejecutar las pruebas de integración ni los benchmarks.

16. Mapa del código y cómo retomarlo en equipo

Qué archivo tocar para cada cambio

Necesidad	Archivos principales
Añadir/validar una acción de navegación	`atlas/contract.py`, `sql/atlas.sql`; revisar generadores y tests.
Cambiar campos GPS o pedido	`atlas/operations.py`, `sql/atlas_operations.sql`, simulador y consultas. Planificar migración.
Cambiar aceptación, deduplicación o persistencia	`atlas/store.py`; comprobar conflictos, reinicios y reintentos.
Cambiar lotes, rutas o fases del publicador	`atlas/server.py`, `atlas/clickhouse.py`.
Cambiar flota, estados o coordenadas	`atlas/simulation.py`; `scripts/operations.py` reutiliza `Scenario`.
Cambiar catálogo y procedencia	`atlas/catalog.py`, tablas `demo_*`; no atribuir precios ficticios a Glovo.
Añadir métrica o visualización	SELECT en `atlas/clickhouse.py`, endpoint en `server.py`, componentes en `web/atlas-dashboard.*`.
Construir miniapp comercial	Nuevos archivos web y `POST /api/events`; catálogo desde `/api/demo-catalog`. Coordinar el enlace con pedidos.
Cambiar compartición	`atlas/share.py`, `api/proxy.js`, `scripts/build_shared.mjs`, `vercel.json`, `compose.share.yaml`.
Revisar carga/consulta histórica	Importador correspondiente, `sql/schema.sql`, `sql/eleme.sql`, `dashboard.py`.

Ruta de incorporación

Jere puede explicar ingesta, SQL y pruebas. Anuar puede empezar por contrato, catálogo y cliente que registra acciones. Echenique puede empezar por las consultas que alimentan el panel, los avisos de demora y la reproducción local. Es una orientación de lectura; el reparto definitivo se acordará cuando os incorporéis.

Antes de modificar una parte, identificar **qué representa su fila**, cómo se genera su ID, qué procedencia tiene y cómo se lee ante reintentos. Probar un cambio pequeño en la ruta local antes de desplegarlo. No hace falta cargar todos los históricos para trabajar sobre Atlas.

Fuentes para profundizar

- Código ejecutable y DDL: `atlas/`, `sql/`, `api/`, Compose y scripts descritos aquí.

- Reproducción: `docs/DEMO-TRACKING.md` y `COMPARTIR-VERCEL.md`.

- Modelo y auditoría: `docs/DATOS-Y-TRACKING.md`, `ESTRUCTURA-BD.md` y `auditoria-esquemas.json`.

- La guía PDF inicial del equipo es anterior al tracking; este documento describe el código revisado el 14/10.

Idea que todo el equipo debe poder explicar: generamos/recibimos eventos, los confirmamos en una cola, publicamos su histórico y estados agregados en ClickHouse, y el dashboard consulta lo que realmente llegó a la base. Los datos públicos demuestran volumen; la simulación demuestra nuestra infraestructura operativa.